List of Notes

Anywhere in your document! 14
You can also add inline notes to your document with inline-note 15





CONFIDENTIAL

Writing in Typst about a long, very scientific topic

Projektarbeit 1 (T3_2000)

as part of the examination for

Bachelor of Science (B.Sc.)

in Computer Science

at the Baden-Württemberg Cooperative State University Karlsruhe

by

John Doe

Erika Musterfrau

Submission Date	10.11.2026
Processing Period	12 Weeks
Matriculation Number, Course	0000000, TINF24B2 1234567, TINF24B1
Training Company	Corp SE Berlin
Department	Human Resources
Supervisor at Training Company	Max Mustermann
Supervisor at University	Heinrich Braun

Acknowledgments

I would like to thank my academic supervisor for their guidance and constructive feedback throughout this thesis. Special thanks go to my company mentor for providing insight into real-world challenges and making my time at the company both productive and enjoyable. Finally, I am grateful to my family and friends for their patience and encouragement.

Abstract

- Deutsch -

Dies ist eine kurze Zusammenfassung, um die Formatierung und den allgemeinen Stil der Vorlage zu zeigen.

Es ist möglich, mehrere Abstracts in unterschiedlichen Sprachen zu haben.

Abstract

- English -

This is a short abstract to show the formatting and general style of the template

It is possible to have multiple abstracts in different languages

Table of Contents

1 Introduction 1
2 Basic Formatting 2
3 Advanced Elements 3
3.1 Figures 3
3.1.1 Image Figures 3
3.1.2 Tables 3
3.1.3 Code Snippets 4
3.2 Math 5
3.3 Block Quotes 6
3.4 Inline Glossary 6
3.5 Notes 7
4 References and Citations 9
4.1 Local Elements 9
4.2 Figures 9
4.3 Bibliography 10
4.4 Acronyms and Glossary 11
4.4.1 Acronyms 11
4.4.2 Glossary 11
5 Reference Management (Recommended: JabRef + BibTeX) 12
5.1 Installing JabRef (macOS via Homebrew) 12
5.2 Opening the Project BibTeX File 12
5.3 Adding References in JabRef 12
5.3.1 Option A (Recommended): Import by DOI 12
5.3.2 Option B: Add an Entry Manually 13
5.3.3 Option C: Import Existing BibTeX 13
5.3.4 Using the Citation Key in Typst 13
5.4 Important: Save Your Changes 13
6 Conclusion 14
7 Visualization 15
7.1 Data-Driven Plots with lilaq 15
7.1.1 Line and Scatter Plots 15
7.1.2 Bar and Stem Plots 17
7.1.3 Error Bars 19
7.1.4 Heatmaps and Contours 20
7.1.5 Vector Fields 21
7.1.6 Boxplot with Hardcoded Values 22
7.1.7 Boxplot from Parsed JSON Data 23
7.2 Diagrams with Mermaid 25
7.2.1 Diagrams as Numbered Figures 25
7.2.2 Class Diagram 26
7.2.3 Flowchart 27
7.2.4 Entity-Relationship Diagram 28

7.2.5 ZenUML Sequence Diagram 29
Bibliography ..... **a**
Abbreviations ..... **b**
Glossary ..... **c**
List of Figures ..... **d**
List of Tables ..... **e**
List of Code ..... **f**
Statutory Declaration ..... **g**
Confidentiality Agreement ..... **h**
AI Acknowledgement ..... **i**
List of Appendices ..... **A**

1 Introduction

This document is an example on how to use the template. The following chapters mark out some specialties of the template and Typst.

You can have a look at every concept by reading the explanation comments or read the source code on GitHub. You will not need every concept explained, so do not despair if you do not understand everything.

If you have any questions, don't hesitate to reach out to the maintainers of this template or create an issue.

2 Basic Formatting

- **Headings and Subheadings:** Create headings by typing `=` followed by your heading text. Increase the number of `=` signs for subheadings, indicating their level within the document structure.
- **Emphasizing Text:** Use `_underscores_` to *italicize words or phrases*, adding emphasis where needed.
- **Lists:** Generate ordered lists with `+` and unordered lists with `-`. Indent lists for sub-items, creating a hierarchical structure.

TABLE I

HEADINGS IN TYPST

Typst Code	Output

`== Main Heading`
`_This_ is a *level 2* heading.`
 	
`=== Subheading`
`+ This is an ordered list.`
`+ It can contain multiple items.`
	2.1 Main Heading
	This is a **level 2** heading.
	2.1.1 Subheading
	1. This is an ordered list.
	2. It can contain multiple items.

3 Advanced Elements

3.1 Figures

Inserting figures and code blocks into your Typst document enhances its informational depth. When specifying a `caption` for a figure, the template will automatically generate a list of figures, making it easy to navigate your document.

Note When using "ieee" Sorting for bibliography, the sources for figures will be evaluated before the text. To prevent "false sorting", you can use `#caption-with-source("Text", [@source])` instead. This will display the caption in outlines without source and will evaluate the source at the time the figure is displayed

3.1.1 Image Figures

TABLE II

IMAGE FIGURES IN TYPST

Typst Code	Output

`#import "@preview/dhbw-oderso:2.4.1": caption-with-source`
`#figure(`
`  image("../assets/placeholder-company-logo.svg"),`
`  caption: caption-with-source("Company Logo", [@electronic]),`
`)`
	
	Figure 1: Company Logo [1]

3.1.2 Tables

Similar to images you can insert table figures. See more table examples and more advanced usage in Appendix 2.

TABLE III

TABLE FIGURES IN TYPST

Typst Code

```
#import "@preview/dhbw-oderso:2.4.1": tablefigure
#tablefigure(
  columns: 3,
  caption: [Example table],
  table-content: (
    table.header([Name], [Age], [Gender]),
    [Alice], [28], [Female],
    [Bob], [34], [Male],
    [Charlie], [22], [Male],
  )
)
```

Output

TABLE IV

EXAMPLE TABLE

Name	Age	Gender
Alice	28	Female
Bob	34	Male
Charlie	22	Male

3.1.3 Code Snippets

This template uses Codly for code snippets. Look at their documentation on how to further customize and control your code blocks.

Code blocks can be created using three backticks:

TABLE V

CODE BLOCKS IN TYPST

Typst Code

````
```js
console.log("Hello World")
```
````

Output

```
console.log("Hello World")
```

You can also wrap a code block in a figure to create a code figure, which will be listed in “List of Code”:

TABLE VI

CODE FIGURES IN TYPST

Typst Code	Output

```
#figure(caption: [My Code])
[```rust
fn main() {
  println!("Hello World!");
}
```]
```

```rust
fn main() {
  println!("Hello World!");
}
```

Code 1: My Code

If you want to create a *normal* figure containing raw text that should not be classified as a code figure (for example command line output), set the `kind` property to `image` explicitly:

TABLE VII

NON-CODE-CODE-FIGURE IN TYPST

Typst Code	Output

```
#figure(caption: [My Command Line
Output], kind: image)[```
> echo "Hello World"
Hello World
```]
```

```
> echo "Hello World"
Hello World
```

Figure 2: My Command Line Output

With `kind: image`, the figure is not listed in "List of Code", but in "List of Figures" instead.

3.2 Math

The math syntax is a loose interpretation of LaTeX, allowing you to create complex mathematical equations with ease. See the Typst documentation for a detailed overview of the math syntax.

TABLE VIII

MATH IN TYPST

Typst Code	Output

`$`
`  sum_(k=0)^n k`
`  &= 1 + ... + n \`
`  &= (n(n+1)) / 2 \`
`$<math-figure>`
	$$\sum_{k=0}^{n} k = 1 + \ldots + n = \frac{n(n+1)}{2} \quad (1)$$

You can reference labeled equations like other figures. (1) shows the sum of the first n natural numbers.

3.3 Block Quotes

TABLE IX

QUOTES IN TYPST

Typst Code	Output

`#quote(`
`  attribution: [Frankling D. Roosevelt @fdr_inaugural_address]`
`)[`
`  The only thing we have to fear is fear itself.`
`]`
	The only thing we have to fear is fear itself.
	— Frankling D. Roosevelt [2]

3.4 Inline Glossary

If you want to include parts of your glossary in your documents content this is possible using the `inline-glossary` function. This is useful if you need word definitions directly in a chapter and want the reader to explicitly read those definitions. The `group`

parameter can be used to only show parts of the glossary. In the following example the *Dependency* group of the example glossary defined in `glossary.typ` is shown. The *Dependency* group contains definitions for Glossarium, Drafting, Codly, Hydra and Linguify, which are the packages used by this template.

TABLE X

INLINE GLOSSARY SHOWING THE TEMPLATES DEPENDENCIES

Typst Code	Output
``` #import "@preview/dhbw-oderso:2.4.1": inline-glossary #import "../glossary.typ": glossary #inline-glossary(glossary, ("Dependencies",), show-all: true) ```	***Codly***: Codly provides styled code blocks with many features like smart indentation, line numbering, highlighting, etc. ***Drafting***: The Drafting package contains functions for content positioning and margin comments/ notes. ***Glossarium***: Glossarium is a simple, easily customizable typst glossary inspired by the LaTeX glossaries package ***Hydra***: Hydra is a Typst package allowing easily displaying the current heading in the page header. ***Linguify***: Linguify brings fluent support to typst, enabling easy i18n.

3.5 Notes

This template uses Drafting for notes. Using `margin-note` you can add notes to the margin of your document.

Anywhere in your document!

You can also add inline notes to your document with `inline-note`

Check out their documentation for more advanced use casese.

You might have noticed the notes listing on the first page of this document. This listing reminds you of the notes still present in your document. Once you remove all notes, the listing will disapear.

4 References and Citations

4.1 Local Elements

You can reference local elements like figures, code blocks, and sections using the `ref()` function. You can also use the synax sugar `<ref>` to define and `@<ref>` to reference references.

TABLE XI
LOCAL ELEMENT REFERENCES IN TYPST

Typst Code	Output

`=== Important Section <section-1>`
`Some important text`
 	
`=== Other Section`
`More important text, just like @section-1`
	4.1.1 Important Section
	Some important text
	4.1.2 Other Section
	More important text, just like Section 4.1.1

4.2 Figures

Add a `<label>` after a figure to reference it using `@label`.

TABLE XII

FIGURE WITH LABEL

Typst Code	Output
`#figure(caption: [Reference me with `reference-figure`], image("../assets/placeholder-company-logo.svg"))<reference-figure>`	 Figure 3: Reference me with `reference-figure`

Figure 3 shows the company logo again.

When using `tablefigure` and `tablefigure-raw` to create a table figure, you can use the `reference` property to reference the created table just like above.

4.3 Bibliography

Typst supports references to external sources, such as books, articles, and websites. You can include a `.yaml` or `.bib` file with your references and use the `cite()` function to reference them in your document. Again, you can use the same syntax sugar `@<ref>` to cite.

TABLE XIII

BIBLIOGRAPHY REFERENCING IN TYPST

Typst Code	Output
`#quote(attribution: [Molly Weasley\ @harry[S.~768]])[ Just Because You're Allowed To Use Magic Now Does Not Mean You Have To Whip Your Wands Out For Everything!]`	Just Because You're Allowed To Use Magic Now Does Not Mean You Have To Whip Your Wands Out For Everything! — Molly Weasley [3, S. 768]

4.4 Acronyms and Glossary

This template supports both acronyms for abbreviations and a glossary for term definitions. Both are implemented using the `glossarium` package, which can be found here: https://typst.app/universe/package/glossarium/.

4.4.1 Acronyms

You can define acronyms in the project configuration. We recommend using this to define abbreviations via the `short` and `long` parameters. Use the `@acr` function to reference an acronym and `@acr:pl` for its plural form. On the first usage, the full long form is displayed. On subsequent usages, only the short form is shown.

All used acronyms will be automatically listed in the "List of Abbreviations".

TABLE XIV
ACRONYMS IN TYPST

Typst Code	Output
`I don't understand @NN or @NN:pl and never will understand.`	I don't understand Neural Network (NN) or NNs and never will understand.

4.4.2 Glossary

For more detailed explanations of technical terms, you can add entries to the glossary in the project configuration. We recommend that you only specify the `long` form and a `description` here, but this is freely customizable.

TABLE XV
GLOSSARY ENTRIES IN TYPST

Typst Code	Output
`When working with @typ, I don't have to reinvent the wheel over and over again.`	When working with Typst, I don't have to reinvent the wheel over and over again.

5 Reference Management (Recommended: JabRef + BibTeX)

The recommended best practice for managing references in this project is to use the graphical citation manager **JabRef** to maintain a **BibTeX (`.bib`) file**.

5.1 Installing JabRef (macOS via Homebrew)

If you are on macOS, install JabRef using Homebrew:

```
brew install --cask jabref
```

Once installed, launch JabRef from your Applications folder or Spotlight.

5.2 Opening the Project BibTeX File

1. Open **JabRef**
2. Go to **File → Open library**
3. Open the file:

```
path_to_your_thesis/template/refs.bib
```

This file serves as the central reference database for the project.

5.3 Adding References in JabRef

5.3.1 Option A (Recommended): Import by DOI

This is the preferred and most reliable method.

1. In JabRef, click **Lookup → Search by DOI**
2. Paste the DOI (e.g., `10.1145/1234567.8901234`)
3. JabRef will automatically fetch the full reference
4. Review the fields and accept the entry

JabRef will automatically generate a **citation key**, which you will use in Typst.

5.3.2 Option B: Add an Entry Manually

Use this only if no DOI is available.

1. Click **Add entry**
2. Choose the entry type (e.g., `Article`, `Book`, `InProceedings`)
3. Fill in the required fields (author, title, year, etc.)
4. Ensure a unique **citation key** is set

5.3.3 Option C: Import Existing BibTeX

If you already have a BibTeX entry:

- Paste it directly into JabRef, or
- Use **File → Import into current library**

5.3.4 Using the Citation Key in Typst

Each reference in JabRef has a **citation key** (e.g., `smith2023example`). Use this key directly in Typst:

```
1 @smith2023example
```

Typst will resolve the citation using `refs.bib`.

5.4 Important: Save Your Changes

After adding or editing references, always save the file in JabRef:

- **File → Save** (or `Cmd + S`)

Changes are not written to `refs.bib` until the file is saved.

6 Conclusion

Happy writing with Typst! If you have any questions, open an issue :)

7 Visualization

Scientific writing lives on good figures. For data-driven plots Typst has **lilaq**, which produces publication-ready plots (line, scatter, bar, histogram, boxplot, heatmaps, vector fields). Use it whenever you plot **numbers**. For schematic diagrams (class diagrams, flowcharts, entity-relationship models) the **merman** package renders Mermaid markup directly during compilation.

The larger plots below are wrapped in `#figure(...)` so they appear in the List of Figures; the small snippets use the `typst-preview` helper to show code and output side by side.

7.1 Data-Driven Plots with lilaq

Every lilaq plot is placed inside an `lq.diagram(...)`, which takes care of axes, labels, title and legend. You then pass one or more plot objects (`lq.plot`, `lq.scatter`, …) as positional arguments. The following sections walk through the most common plot types.

7.1.1 Line and Scatter Plots

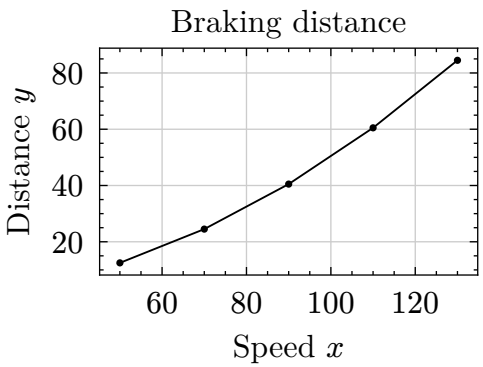
`lq.plot(x, y)` draws a connected line; `lq.scatter(x, y)` draws markers only. The first example is a real quadratic relationship: a car's braking distance grows with the **square** of its speed (roughly $s = \frac{v^2}{200}$). We store the speeds in `x` and the braking distances in `y`, then hand them to `lq.plot`:

TABLE XVI

LINE PLOT WITH LILAQ

Typst Code	Output

```
#import "@preview/lilaq:0.6.0" as lq
#let x = (50, 70, 90, 110, 130)       // speed in km/h
#let y = (12.5, 24.5, 40.5, 60.5, 84.5)  // braking distance in m
#lq.diagram(
  width: 5cm, height: 3cm,
  title: [Braking distance],
  xlabel: [Speed $x$], ylabel: [Distance $y$],
  lq.plot(x, y, color: black),
)
```



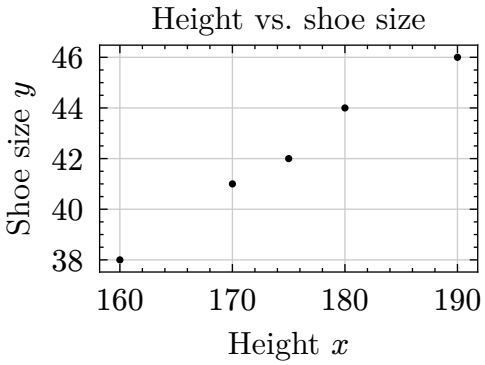
For a scatter plot the idea is the same, but the points need not follow a formula. Here `x` holds the body height of five people and `y` their shoe size, and `lq.scatter` draws one marker per pair:

TABLE XVII

SCATTER PLOT WITH LILAQ

Typst Code	Output

```
#import "@preview/lilaq:0.6.0" as lq
#let x = (160, 170, 175, 180, 190)  // height in cm
#let y = (38, 41, 42, 44, 46)       // shoe size
#lq.diagram(
  width: 5cm, height: 3cm,
  title: [Height vs. shoe size],
  xlabel: [Height $x$], ylabel: [Shoe size $y$],
  lq.scatter(x, y, color: black),
)
```



7.1.2 Bar and Stem Plots

`lq.bar` is ideal for categorical data, `lq.stem` for discrete sequences. The bar example compares the revenue of four quarters: `x` numbers the quarters, `y` holds the revenue in thousands of euro.

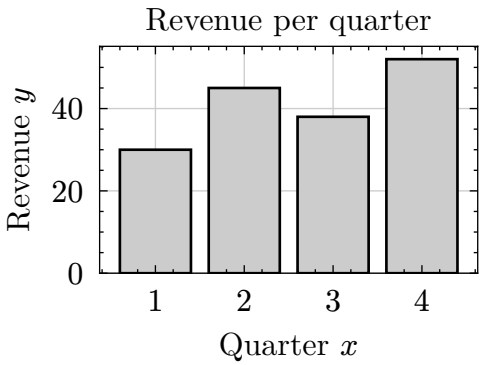
TABLE XVIII

Bar Plot with lilaq

Typst Code

```
#import "@preview/lilaq:0.6.0" as lq
#let x = (1, 2, 3, 4)            // quarter
#let y = (30, 45, 38, 52)        // revenue in k€
#lq.diagram(
  width: 5cm, height: 3cm,
  title: [Revenue per quarter],
  xlabel: [Quarter $x$], ylabel: [Revenue $y$],
  lq.bar(x, y, fill: luma(80%), stroke: black),
)
```

Output



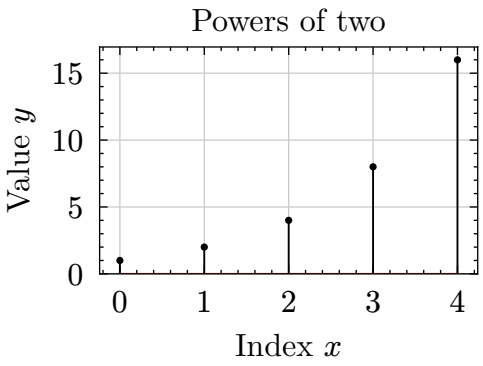
The stem plot below shows a discrete sequence: `x` is the index and `y` the corresponding power of two ($y = 2^x$).

TABLE XIX

STEM PLOT WITH LILAQ

Typst Code	Output

```
#import "@preview/lilaq:0.6.0" as lq
#let x = (0, 1, 2, 3, 4)
#let y = (1, 2, 4, 8, 16)        // powers of two
#lq.diagram(
  width: 5cm, height: 3cm,
  title: [Powers of two],
  xlabel: [Index $x$], ylabel: [Value $y$],
  lq.stem(x, y, color: black),
)
```



7.1.3 Error Bars

Uncertainties are attached directly to `lq.plot` via `yerr` (or `xerr`). The error can be a single constant or an array with one value per data point. In this example `x` is the day, `y` the measured temperature, and `err` the measurement uncertainty of each value:

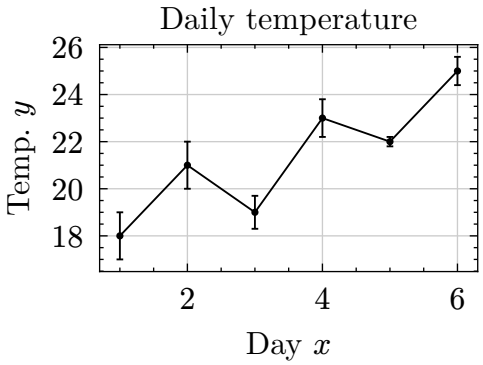
TABLE XX

Error Bars with lilaq

Typst Code

```
#import "@preview/lilaq:0.6.0" as lq
#let x = (1, 2, 3, 4, 5, 6)              // day
#let y = (18, 21, 19, 23, 22, 25)        // temperature in °C
#let err = (1, 1, .7, .8, .2, .6)        // uncertainty
#lq.diagram(
  width: 5cm, height: 3cm,
  title: [Daily temperature],
  xlabel: [Day $x$], ylabel: [Temp. $y$],
  lq.plot(x, y, yerr: err, color: black),
)
```

Output



7.1.4 Heatmaps and Contours

For data over a 2D grid, `lq.colormesh(x, y, z)` draws a heatmap and `lq.contour(x, y, z)` draws level curves. `z` may be an explicit grid or a function `(x, y) => value`.

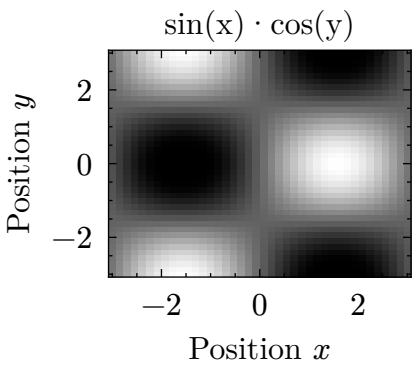
TABLE XXI

COLORMESH WITH LILAQ

Typst Code

```
#import "@preview/lilaq:0.6.0" as lq
// z = sin(x)·cos(y) over a 40×40 grid.
// `map` overrides lilaq's default Viridis colormap with a black-to-white
// ramp so the heatmap stays legible in grayscale print (IEEE requirement:
// no information encoded through color/hue alone).
#lq.diagram(
  width: 4cm, height: 3cm,
  title: [sin(x)·cos(y)],
  xlabel: [Position $x$],
  ylabel: [Position $y$],
  lq.colormesh(
    lq.linspace(-3, 3, num: 40),
    lq.linspace(-3, 3, num: 40),
    (x, y) => calc.sin(x) * calc.cos(y),
    map: (luma(0%), luma(100%)),
  ),
)
```

Output



7.1.5 Vector Fields

`lq.quiver(x, y, directions)` draws a field of arrows. The `directions` argument is a function returning a `(dx, dy)` vector for each grid point.

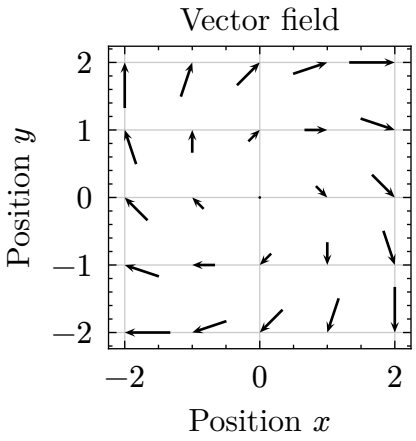
TABLE XXII

VECTOR FIELD WITH LILAQ

Typst Code

```
#import "@preview/lilaq:0.6.0" as lq
// Rotating field: each arrow points (x+y, y-x)
#lq.diagram(
  width: 4cm, height: 4cm,
  title: [Vector field],
  xlabel: [Position $x$],
  ylabel: [Position $y$],
  lq.quiver(
    lq.arange(-2, 3),
    lq.arange(-2, 3),
    (x, y) => (x + y, y - x),
  ),
)
```

Output



7.1.6 Boxplot with Hardcoded Values

A **boxplot** (box-and-whisker plot) summarizes a dataset's distribution using five statistics: the minimum and maximum (whisker endpoints, excluding outliers), the quartiles **Q1** and **Q3** (box edges), and the **median** (line inside the box). Any value further than $1.5 \cdot \text{IQR}$ (interquartile range $= \text{Q3} - \text{Q1}$) from the box is drawn as an individual outlier point.

In the simplest case, the data is written directly into the source file. Here the data is the exam scores (out of 100) of a class. One student clearly underperformed at 30 points, which shows up as an outlier below the whisker.

Rather than assembling an `lq.diagram` by hand, this template provides a `boxplot-figure` helper. It renders the boxplot as a numbered, referenceable figure in the black-and-white IEEE palette (white box, black outline, median, whiskers and outliers), so distributions stay legible in grayscale print. You pass the data directly

through `values`. The preview below shows the underlying `lq.boxplot` call next to its rendered output:

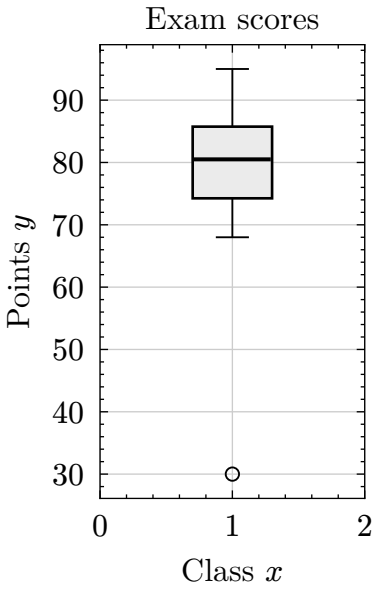
TABLE XXIII

BOXPLOT WITH LILAQ (IN-LINE VALUES)

Typst Code

```
#import "@preview/lilaq:0.6.0" as lq
#let scores = (82, 75, 88, 91, 68, 79, 85, 72, 95, 30, 77, 84)
#lq.diagram(
  width: 3.5cm, height: 6cm,
  title: [Exam scores],
  xlabel: [Class $x$], ylabel: [Points $y$],
  xlim: (0, 2),
  lq.boxplot(scores, x: 1, width: 0.6,
    fill: luma(92%), stroke: black,
    median: 1.5pt + black, whisker: black,
    cap: black, outlier-stroke: black),
)
```

Output



7.1.7 Boxplot from Parsed JSON Data

Hardcoding data into the document is fine for a quick example, but real datasets usually live in a separate data file. Typst has a **built-in** `json()` function (no extra package needed) that reads a `.json` file and converts it into Typst values: objects become dictionaries and arrays become arrays. Related built-ins are `csv()`, `toml()` and `yaml()`; `json.decode` parses a JSON **string** instead of a file.

The file `assets/data/boxplot-sample.json` holds the scores of a second exam:

```
{
  "exam": "Math Exam B",
  "scores": [82, 75, 88, 91, 68, 79,
             85, 72, 95, 30, 77, 84]
}
```

Figure 4: The JSON data file

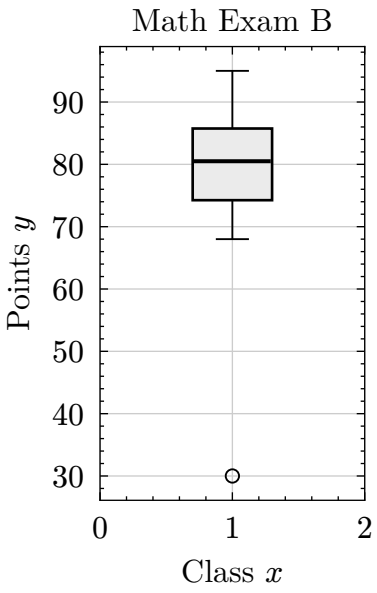
We read the file with Typst's built-in `json()` and feed the `scores` array straight into `lq.boxplot`. Paths are resolved relative to the file doing the reading, which lives in `chapters/`, so we go up one level with `../assets/...`. The preview shows the loading code next to its output:

TABLE XXIV

Boxplot with lilaq (parsed from JSON)

Typst Code	Output

```
#import "@preview/lilaq:0.6.0" as lq
#let data = json("../assets/data/boxplot-sample.json")
#lq.diagram(
  width: 3.5cm, height: 6cm,
  title: [#data.exam],
  xlabel: [Class $x$], ylabel: [Points $y$],
  xlim: (0, 2),
  lq.boxplot(data.scores, x: 1, width: 0.6,
    fill: luma(92%), stroke: black,
    median: 1.5pt + black, whisker: black,
    cap: black, outlier-stroke: black),
)
```



If you need the raw data elsewhere, you can still load it yourself: Typst's `json()` returns a dictionary you access like any other. `data.exam` is the title string and `data.scores` the array of numbers.

```
#let data = json("../assets/data/boxplot-sample.json")
Exam *#data.exam* has #data.scores.len() scores.
```

Figure 5: Loading data from JSON

To compare several groups, pass one `lq.boxplot` per group with a distinct `x` position: `xticks` labels each box and `xlim` keeps them evenly spaced. The block below is commented out. Remove the `/*` and `*/` to render a second boxplot that compares the exam scores of three parallel classes:

7.2 Diagrams with Mermaid

Mermaid is a text-based diagram language: you describe a diagram in a few lines of markup and the renderer lays it out for you. The `merman` package ships a WebAssembly renderer, so the diagrams are produced during compilation, with no browser or Node.js required. A single function call renders any Mermaid source:

```
#import "@preview/merman:0.1.0": mermaid
#mermaid("flowchart TD
  A[Start] --> B[Stop]", background: none)
```

Passing `background: none` keeps the diagram transparent so it blends into the page. The helper used below shows the Mermaid source on the left and the rendered diagram on the right.

7.2.1 Diagrams as Numbered Figures

A bare `#mermaid(...)` call drops the diagram into the page as a loose image: it has no figure number, no entry in the "List of Figures", and cannot be cross-referenced. For a scientific thesis, every diagram should instead be a proper figure so it is numbered, listed, and referenceable like any other illustration. The `mermaid-figure` helper wraps the diagram accordingly:

```
#import "visualization-helpers.typ": mermaid-figure
```

```
#mermaid-figure(
  [Build pipeline stages],
  "flowchart LR
    A[Compile] --> B[Test]
    B --> C[Deploy]",
  label: <fig-pipeline>,
)
```

This produces a numbered figure you can reference with its label — for example the flowchart in Table XXVI below. Pass an optional `source: [@key]` to add a citation after the caption (routed through `caption-with-source` so it does not disturb IEEE outline sorting), and `width` to control the rendered size.

By default `mermaid-figure` renders in **black and white** rather than Mermaid's colored theme. This matches scientific conventions: IEEE figures, for example, must stay legible in grayscale and must never encode information through color alone. Pass `theme: none` for Mermaid's default colors, or your own theme variable map to override individual values.

The side-by-side previews below show the Mermaid syntax next to its output for learning purposes; in an actual thesis, prefer `mermaid-figure` so each diagram is a proper figure.

7.2.2 Class Diagram

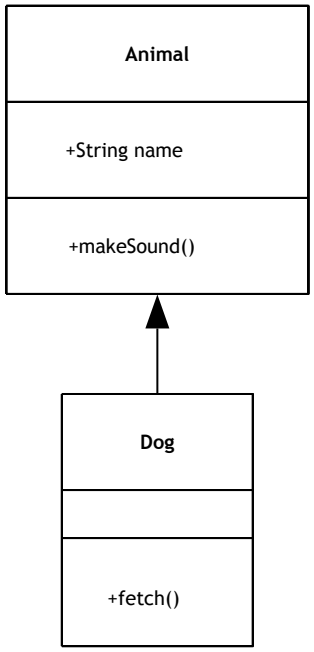
A `classDiagram` shows classes with their attributes and methods, plus the relationships between them. The arrow `<|--` means "inherits from": a `Dog` **is an** `Animal`.

TABLE XXV

Class Diagram with Mermaid: inheritance between Animal and Dog

Mermaid Markup	Output

```
classDiagram
  class Animal {
    +String name
    +makeSound()
  }
  class Dog {
    +fetch()
  }
  Animal <|-- Dog
```



7.2.3 Flowchart

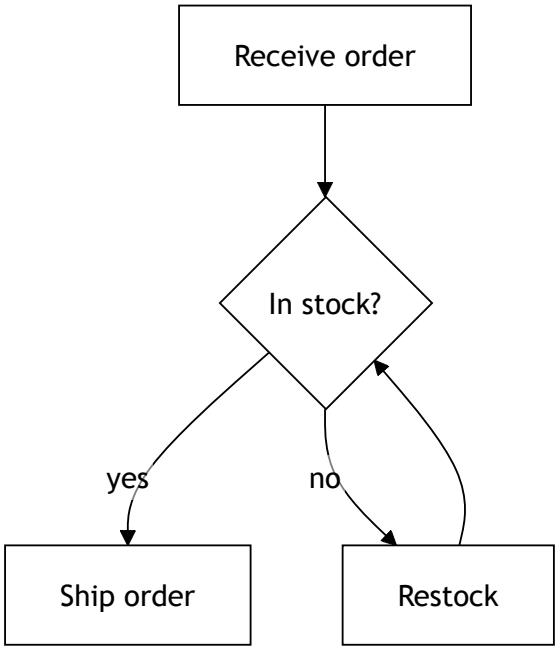
A `flowchart` describes a process. `TD` lays it out top-down; `[...]` is a step, `{...}` a decision, and the arrow labels (`|yes|`) annotate the branches.

TABLE XXVI

Flowchart with Mermaid: order handling with a stock-check decision

Mermaid Markup	Output

```
flowchart TD
  A[Receive order] --> B{In stock?}
  B -->|yes| C[Ship order]
  B -->|no| D[Restock]
  D --> B
```



7.2.4 Entity-Relationship Diagram

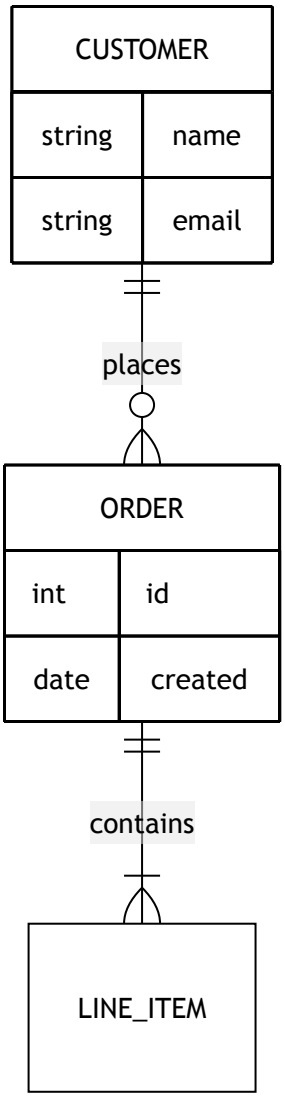
An `erDiagram` models how database entities relate. The crow's-foot notation `||--o{` reads "one customer places zero or more orders".

TABLE XXVII

Entity-Relationship Diagram with Mermaid: customers, orders and line items

Mermaid Markup	Output

```
erDiagram
  CUSTOMER ||--o{ ORDER : places
  ORDER ||--|{ LINE_ITEM : contains
  CUSTOMER {
    string name
    string email
  }
  ORDER {
    int id
    date created
  }
```



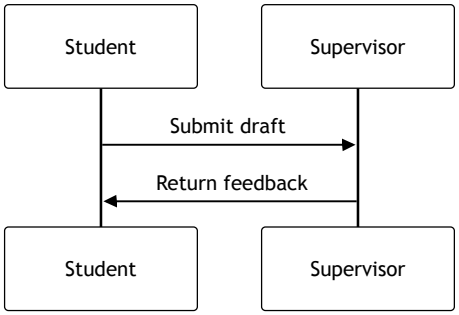
7.2.5 ZenUML Sequence Diagram

Mermaid also supports ZenUML, a compact way to write sequence diagrams. Each line is one message from a sender to a receiver.

TABLE XXVIII

ZenUML Sequence Diagram with Mermaid: draft submission and feedback

Mermaid Markup	Output

`zenuml`
`  Student->Supervisor: Submit draft`
`  Supervisor->Student: Return feedback`
	

Bibliography

[1] Ishkur, “Ishkur's Guide to Electronic Music.” [Online]. Available: http://www.techno.org/electronic-music-guide/

[2] “Franklin D. Roosevelt's First Inaugural Address.” [Online]. Available: https://avalon.law.yale.edu/20th_century/froos1.asp

[3] J. K. Rowling, *Harry Potter and the Order of the Phoenix*, vol. 5. 2003, p. 768.

Abbreviations

***NN* – Neural Network** 11

Glossary

Typst: Typst is a new markup-based typesetting system that is designed to be as powerful as LaTeX while being much easier to learn and use. 11

Dependencies

Codly: Codly provides styled code blocks with many features like smart indentation, line numbering, highlighting, etc. 7

Drafting: The Drafting package contains functions for content positioning and margin comments/notes. 7

Glossarium: Glossarium is a simple, easily customizable typst glossary inspired by the LaTeX glossaries package 7

Hydra: Hydra is a Typst package allowing easily displaying the current heading in the page header. 7

Linguify: Linguify brings fluent support to typst, enabling easy i18n. 7

List of Figures

Figure 1 Company Logo 3
Figure 2 My Command Line Output 5
Figure 3 Reference me with `reference-figure` 10
Figure 4 The JSON data file 24
Figure 5 Loading data from JSON 25

List of Tables

Table I Headings in Typst 2
Table II Image Figures in Typst 3
Table III Table Figures in Typst 4
Table IV Example table 4
Table V Code Blocks in Typst 4
Table VI Code Figures in Typst 5
Table VII Non-Code-Code-Figure in Typst 5
Table VIII Math in Typst 6
Table IX Quotes in Typst 6
Table X Inline Glossary Showing the Templates Dependencies 7
Table XI Local Element References in Typst 9
Table XII Figure with Label 10
Table XIII Bibliography Referencing in Typst 10
Table XIV Acronyms in Typst 11
Table XV Glossary Entries in Typst 11
Table XVI Line Plot with lilaq 16
Table XVII Scatter Plot with lilaq 17
Table XVIII Bar Plot with lilaq 18
Table XIX Stem Plot with lilaq 19
Table XX Error Bars with lilaq 20
Table XXI Colormesh with lilaq 21
Table XXII Vector field with lilaq 22
Table XXIII Boxplot with lilaq (in-line values) 23
Table XXIV Boxplot with lilaq (parsed from JSON) 24
Table XXV Class Diagram with Mermaid: inheritance between Animal and Dog 27
Table XXVI Flowchart with Mermaid: order handling with a stock-check decision 28
Table XXVII Entity-Relationship Diagram with Mermaid: customers, orders and line items 29
Table XXVIII ZenUML Sequence Diagram with Mermaid: draft submission and feedback 30

List of Code

Code 1 My Code 5

Statutory Declaration

- Deutsch -

Wir versichern hiermit, dass wir die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet haben und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt haben und diese bislang nicht veröffentlicht wurde.
Wir haben bei der Erstellung der Arbeit KI-Werkzeuge eingesetzt. Dies haben wir an entsprechender Stelle der Arbeit kenntlich gemacht.



Karlsruhe	10.11.2026	
Place	Date	Signature

Karlsruhe	10.11.2026	Erika Musterfrau
Place	Date	Signature

Confidentiality Agreement

The contents of this work may not be made accessible, in whole or in part, to persons outside the examination process and the evaluation procedure, unless otherwise authorized by the Dual Partner.

AI Acknowledgement

Tool	Usage Description
ChatGPT	1. Vibed chapter 1 - 6 2. Grammer correction

List of Appendices

Appendix 1 Relevant Stuff ..... B
Appendix 2 Table Examples C

Appendix 1 Relevant Stuff

Appendix 1.1 This is some more source code

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do.

You can reference this appendix using `@appendix-relevant-stuff`.

Appendix 2 Table Examples

This appendix shows example tables. Sourced from https://ieeexplore.ieee.org/document/7165621.

TABLE XXIX
RESEARCH QUESTION

RQ	Research Question	Motivation
1	Are there FPA problems being reported in terms of accuracy?	Identify and summarize potential FPA problems, pointed out by researchers.
2	Are there proposals to improve the FPA's accuracy? Which improvements are being proposed?	Identify and summarize the approaches proposed to improve the FPA accuracy.
3	Are the proposed improvements effective?	Verify if the proposed improvements are more effective when compared to the standard FPA.
4	What are the limitations for the proposed improvements?	Identify potential problems or weaknesses for the proposed improvements.

TABLE XXX
DATA FUNCTION COMPLEXITY

		DET		
		1-19	20-50	>50
Record	1	Low	Low	Average
Element	2-5	Low	Average	High
Types (RET)	>5	Average	High	High

TABLE XXXI
CUSTOM TABLE

A	custom
table	style

A	fixed
styled	table
without	figure-wrapper